

Les boucles dans Thymeleaf

1. La boucle `th:each`

Présentation

Thymeleaf fournit une seule structure de boucle : **th:each**. Elle itère sur :

- les objets implémentant `java.lang.Iterable` (`List`, `Set`, ...);
- les objets de type `java.util.Map` (chaque élément est alors un `Map.Entry`);
- les tableaux Java;
- les objets `java.util.Iterator` et `java.util.Enumeration`.

Syntaxe la plus simple :

```
<someHtmlTag th:each="item : ${items}">
    ...
</someHtmlTag>
```

La balise **<th:block>** est une balise « virtuelle » : elle n'existe pas en HTML et disparaît du résultat final. Elle est utile pour répéter plusieurs éléments à la fois, ou lorsqu'on ne veut pas ajouter de balise HTML supplémentaire :

```
<th:block th:each="item : ${items}">
    ...
</th:block>
```

Premier exemple

Contrôleur :

```
@GetMapping("/loop-simple-example")
public String loopSimpleExample(Model model) {
    String[] flowers = {"Rose", "Lily", "Tulip", "Carnation", "Hyacinth"};
    model.addAttribute("flowers", flowers);
    return "loop-simple-example";
}
```

Template `loop-simple-example.html` :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8" />
    <title>Boucle</title>
</head>
<body>
```

```
<h1>th:each</h1>
<ul>
  <th:block th:each="flower : ${flowers}">
    <li th:text="${flower}">..</li>
  </th:block>
</ul>
</body>
</html>
```

Résultat (HTML produit) :

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8" />
  <title>Boucle</title>
</head>
<body>
  <h1>th:each</h1>
  <ul>
    <li>Rose</li>
    <li>Lily</li>
    <li>Tulip</li>
    <li>Carnation</li>
    <li>Hyacinth</li>
  </ul>
</body>
</html>
```

Ici, le `<th:block>` n'était pas indispensable : on aurait pu écrire directement `<li th:each="flower : ${flowers}" th:text="${flower}">..</li>`. Il devient utile dès qu'on répète plusieurs éléments frères.

2. La variable d'état

La syntaxe complète de `th:each` comporte deux variables : la **variable d'élément** (*item variable*) et la **variable d'état** (*status variable*).

```
<someHtmlTag th:each="item, iState : ${items}">
  ...
</someHtmlTag>
```

La variable d'état fournit des informations sur l'itération en cours :

Propriété	Description
index	Indice de l'itération en cours, à partir de 0
count	Rang de l'itération en cours, à partir de 1 (index + 1)
size	Nombre total d'éléments

Propriété	Description
current	Élément courant (même valeur que la variable d'élément)
even / odd	Rang pair ou impair, compté à partir de 1 : la première ligne est donc <i>odd</i> (comme les sélecteurs CSS :nth-child(odd) et :nth-child(even))
first	Vrai pour la première itération
last	Vrai pour la dernière itération

Si l'on ne nomme pas la variable d'état, Thymeleaf la crée sous le nom de la variable d'élément suivi de Stat (par exemple flowerStat pour th:each="flower : \${flowers}").

Exemple complet

```
@GetMapping("/loop-example")
public String loopExample(Model model) {
    String[] flowers = {"Rose", "Lily", "Tulip", "Carnation", "Hyacinth"};
    model.addAttribute("flowers", flowers);
    return "loop-example";
}

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8" />
    <title>Boucle</title>
    <style>table th, table td { padding: 5px; }</style>
</head>
<body>
    <h1>th:each</h1>
    <table border="1">
        <tr>
            <th>index</th><th>count</th><th>size</th><th>even</th>
            <th>odd</th><th>first</th><th>last</th><th>Fleur</th>
        </tr>
        <tr th:each="flower, state : ${flowers}">
            <td th:text="${state.index}">index</td>
            <td th:text="${state.count}">count</td>
            <td th:text="${state.size}">size</td>
            <td th:text="${state.even}">even</td>
            <td th:text="${state.odd}">odd</td>
            <td th:text="${state.first}">first</td>
            <td th:text="${state.last}">last</td>
            <td th:text="${flower}">Nom de la fleur</td>
        </tr>
    </table>
</body>
</html>
```

Résultat : un tableau de cinq lignes. Pour la première, index vaut 0, count vaut 1, first vaut true et odd vaut true ; pour la dernière, index vaut 4, count vaut 5, last vaut true et even vaut false.

3. Exemples complémentaires

La classe Person

```
package com.example.thymeleaf.model;

public class Person {

    private Long id;
    private String fullName;
    private String email;

    public Person() { }

    public Person(Long id, String fullName, String email) {
        this.id = id;
        this.fullName = fullName;
        this.email = email;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

Remarque. Depuis Java 16, on peut aussi définir cette classe avec un record. Dans ce cas, les accesseurs n'étant plus nommés getX(), on écrit person.fullName() dans les expressions plutôt que person.fullName.

th:each avec une List

```
@GetMapping("/loop-list-example")
public String loopListExample(Model model) {
    List<Person> people = List.of(
        new Person(1L, "Tom", "tom@example.com"),
        new Person(2L, "Jerry", "jerry@example.com"),
        new Person(3L, "Donald", "donald@example.com"));

    model.addAttribute("people", people);
}
```

```

    return "loop-list-example";
}

<head>
  <meta charset="UTF-8" />
  <title>Boucle</title>
  <style>
    table th, table td { padding: 5px; }
    .row      { font-style: italic; }
    .even-row { color: black; }
    .odd-row  { color: blue; }
  </style>
</head>
<body>
  <h1>th:each</h1>
  <table border="1">
    <tr>
      <th>N°</th>
      <th>Nom complet</th>
      <th>E-mail</th>
    </tr>
    <tr th:each="person, state : ${people}"
        class="row"
        th:classappend="${state.odd} ? 'odd-row' : 'even-row'">
      <td th:text="${state.count}">N°</td>
      <td th:text="${person.fullName}">Nom complet</td>
      <td th:text="${person.email}">E-mail</td>
    </tr>
  </table>
</body>

```

th:classappend **ajoute** une classe à l'attribut class existant, alors que th:class le remplace. La ligne 1 reçoit odd-row (bleue), la ligne 2 even-row (noire), et ainsi de suite.

th:each avec une Map

Lorsqu'on itère sur une Map, chaque élément est un objet Map.Entry : on accède à la clé avec .key et à la valeur avec .value.

```

@GetMapping("/loop-map-example")
public String loopMapExample(Model model) {
    // Clé : numéro de téléphone. LinkedHashMap conserve l'ordre d'insertion.
    Map<String, Person> contacts = new LinkedHashMap<>();
    contacts.put("110033", new Person(1L, "Tom", "tom@example.com"));
    contacts.put("110055", new Person(2L, "Jerry", "jerry@example.com"));
    contacts.put("110077", new Person(3L, "Donald", "donald@example.com"));

    model.addAttribute("contacts", contacts);
    return "loop-map-example";
}

```

```
<table border="1">
  <tr>
    <th>N°</th>
    <th>Téléphone</th>
    <th>Nom complet</th>
    <th>E-mail</th>
  </tr>
  <tr th:each="mapItem, state : ${contacts}"
      class="row"
      th:classappend="${state.odd} ? 'odd-row' : 'even-row'">
    <td th:text="${state.count}">N°</td>
    <td th:text="${mapItem.key}">Téléphone</td>
    <td th:text="${mapItem.value.fullName}">Nom complet</td>
    <td th:text="${mapItem.value.email}">E-mail</td>
  </tr>
</table>
```

Attention à l'ordre. Un HashMap ne garantit aucun ordre d'itération : l'affichage peut différer de l'ordre d'insertion. Utilisez un LinkedHashMap (ordre d'insertion) ou un TreeMap (ordre des clés) pour obtenir un affichage prévisible.

Créer une liste directement dans le template

```
<th:block th:with="flowers = ${ {'Rose', 'Lily', 'Tulip'} }">
  <table border="1">
    <tr>
      <th>N°</th>
      <th>Fleur</th>
    </tr>
    <tr th:each="flower, state : ${flowers}">
      <td th:text="${state.count}">N°</td>
      <td th:text="${flower}">Fleur</td>
    </tr>
  </table>
</th:block>
```

L'expression `{ 'Rose', 'Lily', 'Tulip' }` crée une List en SpringEL. Cette technique convient pour de petites listes de constantes ; pour de vraies données, préparez-les dans le contrôleur.

4. Bonnes pratiques

- **Gérer la liste vide.** Combinez `th:if` avec `#lists.isEmpty` (ou `#maps.isEmpty`, `#sets.isEmpty`) pour afficher un message plutôt qu'un tableau vide :

```
<p th:if="${#lists.isEmpty(people)}">Aucune personne à afficher.</p>
<table th:unless="${#lists.isEmpty(people)}">...</table>
```

- **Priorité des attributs.** Sur une même balise, `th:each` est évalué avant `th:if` : le test peut donc utiliser la variable d'élément.

```
<li th:each="p : ${people}" th:if="${p.email != null}"  
th:text="${p.fullName}">...</li>
```

- **Séparateurs.** Utilisez `state.last` pour éviter une virgule finale : `<span th:text="${p.fullName} + (${state.last} ? ' : ', ' )">`.
- **Volume de données.** Une boucle génère tout le HTML d'un coup : pour des milliers de lignes, paginez côté serveur (Pageable de Spring Data) plutôt que d'envoyer toute la liste au template.

Résumé

- `th:each` est la seule boucle de Thymeleaf ; elle accepte `Iterable`, `Map`, tableaux, `Iterator` et `Enumeration`.
- `<th:block>` permet de répéter plusieurs éléments sans balise HTML supplémentaire.
- La variable d'état expose `index` (à partir de 0), `count` (à partir de 1), `size`, `current`, `even`, `odd`, `first` et `last`.
- `even` et `odd` se comptent à partir de 1 : la première ligne est *odd*.
- Pour une `Map`, chaque élément est un `Map.Entry` (`.key`, `.value`) ; préférez `LinkedHashMap` pour un ordre stable.
- Utilisez `th:text` (et non `th:utext`) pour échapper le contenu affiché.

Exercices proposés

1. Afficher une liste de personnes dans un tableau, en colorant en gris les lignes paires, puis en mettant en gras la dernière ligne grâce à `state.last`.
2. Afficher les noms séparés par des virgules, sans virgule après le dernier.
3. Itérer sur une `Map<String, List<Person>>` (groupes de personnes) avec deux boucles imbriquées.
4. Expliquer pourquoi `<li th:each="p : ${people}" th:if="${p.email != null}">` fonctionne, alors que `<li th:with="1 = ${people}" th:each="p : ${1}">` ne fonctionne pas (voir le document sur les variables).
5. Remplacer l'ensemble de la liste par une page de 10 éléments avec `Pageable` et des liens « précédent / suivant ».